CS 5330
Project 3 Report: Real-time 2-D Object Recognition
Submitted By: . CHARUL - 002535330

Project Description

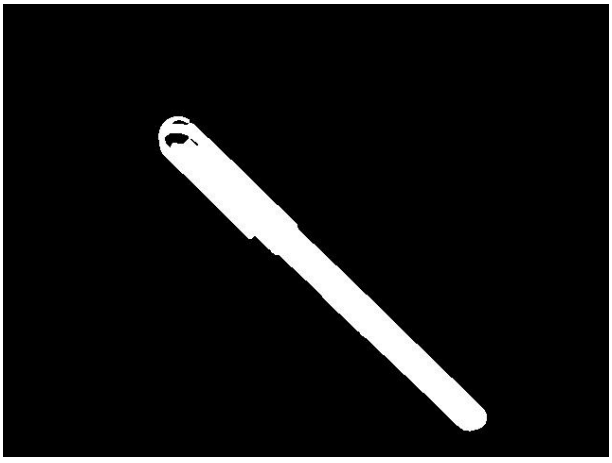
This project is a real-time object recognition system built in C++ with OpenCV. You point a camera at a dark object on a white background and the system figures out what it is. It works through a pipeline: first it thresholds the video to separate the object from the background, then cleans up the binary image with morphological filtering, then finds connected regions, computes shape features for each region, and finally classifies the object by comparing those features to a database of known objects.

I trained it on 6 objects: glasses, keys, a pen, a remote, scissors, and a watch. The whole thing runs live in one window that shows all the stages side by side, with the predicted label drawn right on top of each object.

I also added a second classifier using a ResNet18 deep network, which recognizes objects from their image embeddings instead of hand-built features. The features I designed myself (percent filled, aspect ratio, Hu moment) are all rotation, scale, and translation invariant, so you can move the object around or spin it and the system still recognizes it.

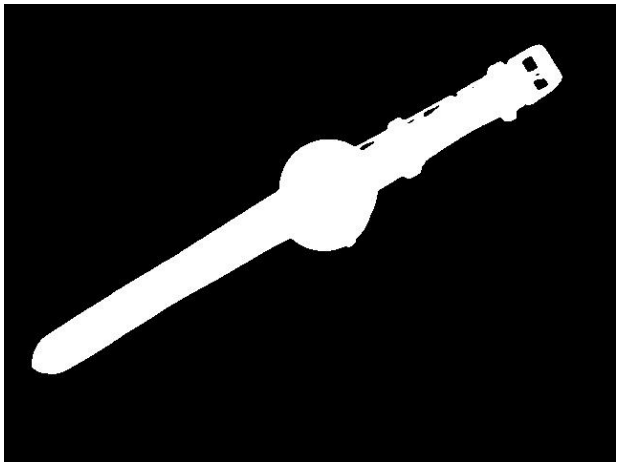
Task 1: Threshold the input video

Original Image 1.1



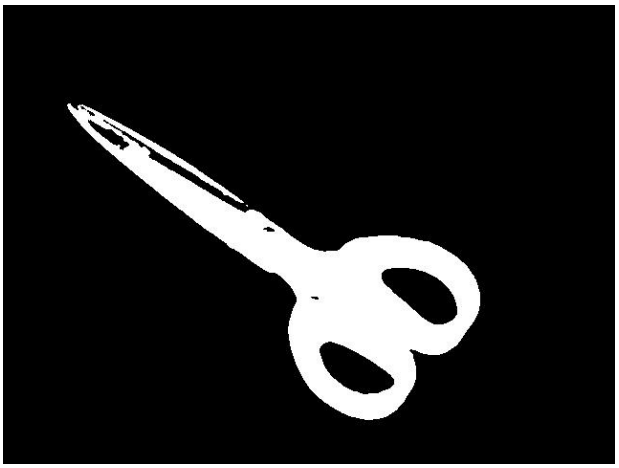
Thresholded Image 1.1

Original Image 1.2



Thresholded Image 1.2

Original Image 1.3



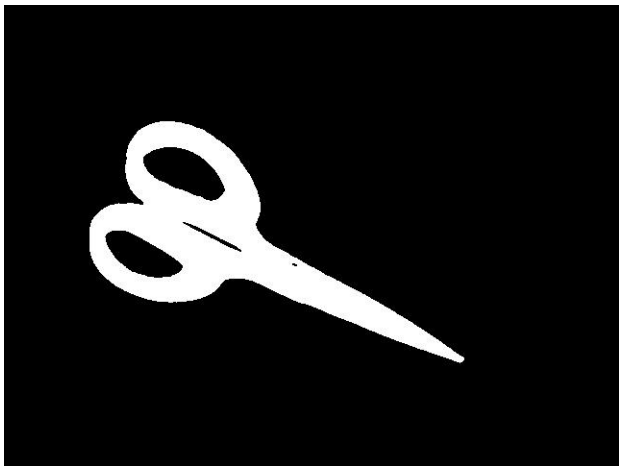
Thresholded Image 1.3

Required images Task 1: Original Images & Threshold Images

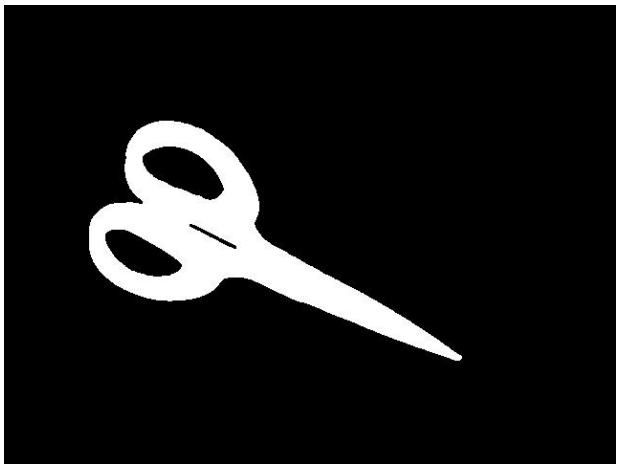
I separate objects from background using a dynamic threshold computed with the ISODATA algorithm (K-means with K=2), written from scratch. The image is first blurred slightly for uniformity, then strongly saturated pixels are darkened in HSV space so colored objects move away from the white background. ISODATA samples a fraction of pixels, splits them into two intensity clusters, and sets the threshold at the midpoint of the two cluster means, so it adapts to lighting automatically rather than using a fixed value.

Task 2: Clean up the binary image

Original Image 2.1

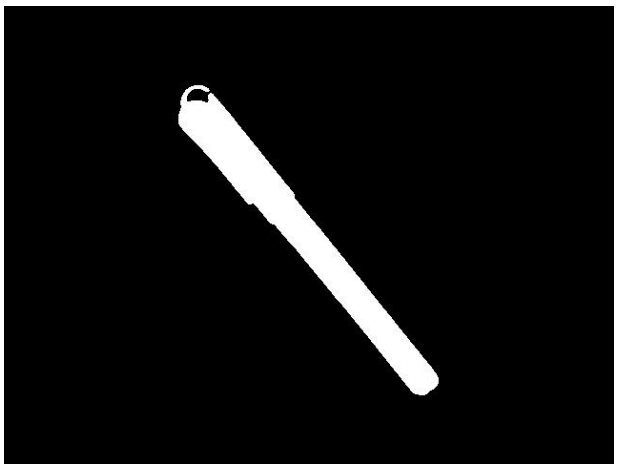


Thresholded Image 2.1

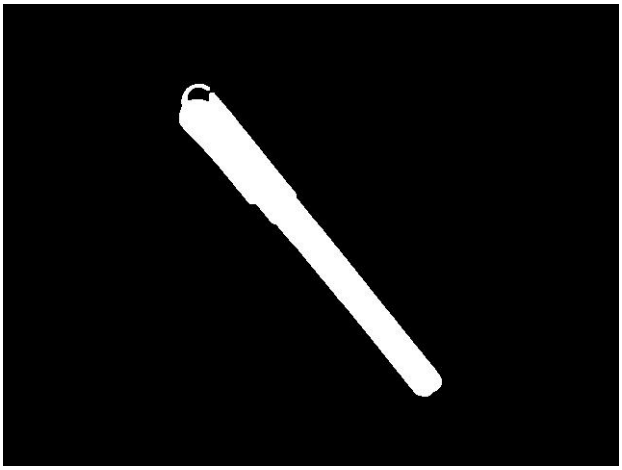


Cleaned Image 2.1

Original Image 2.2

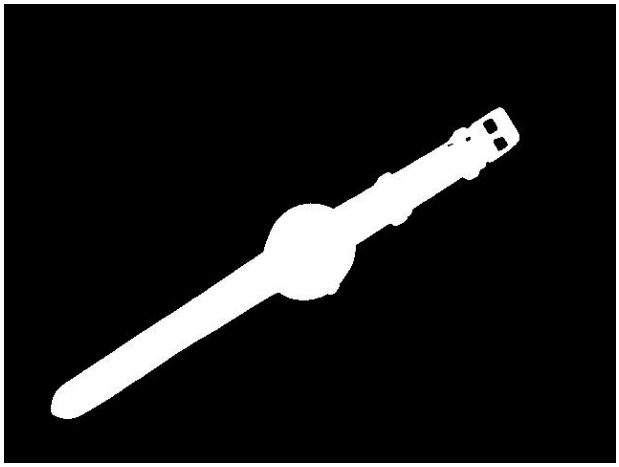


Thresholded Image 2.2

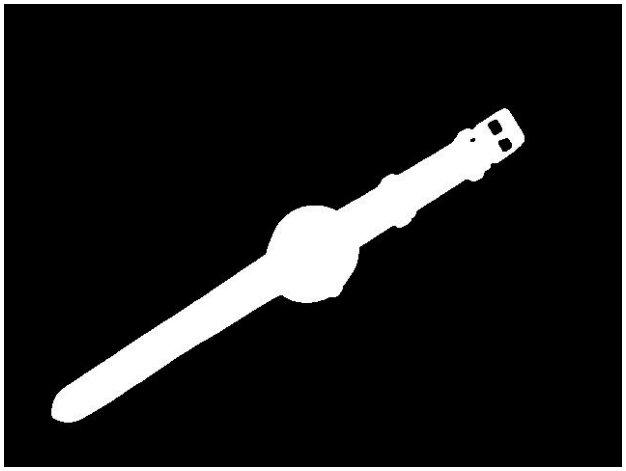


Cleaned Image 2.2

Original Image 2.3



Thresholded Image 2.3



Cleaned Image 2.3

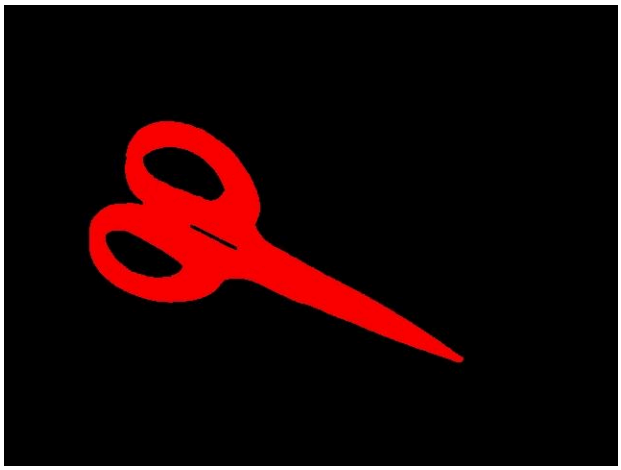
Required images Task 2: Original Images, Threshold Images & Cleaned Images

I clean the binary image with morphological filtering written from scratch (erosion and dilation with a 3x3 structuring element). My cleanup runs one round of opening (erode->dilate) to

remove background speckle, then two rounds of closing (dilate->erode) to fill internal holes. Opening runs first, so noise isn't grown into permanent blobs; closing runs more aggressively because the main issue in my images was gaps inside reflective surfaces (like the scissor blades).

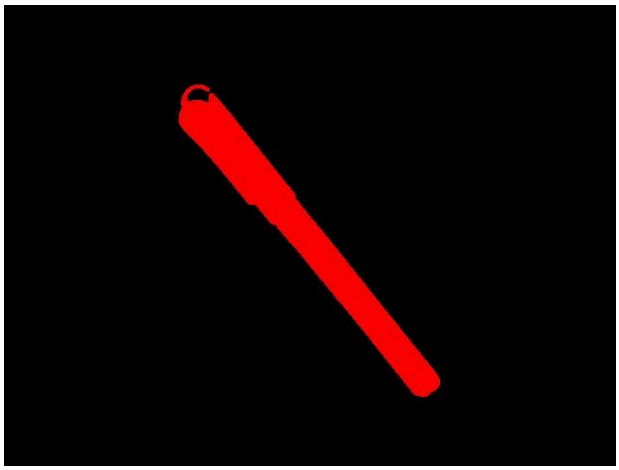
Task 3: Segment the image into regions

Original Image 3.1



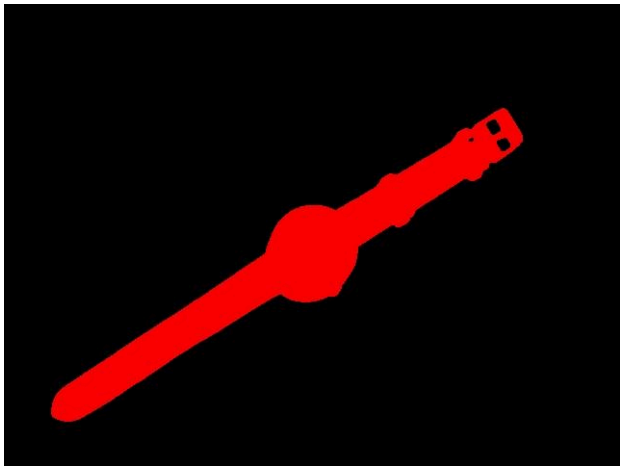
Region Map 3.1

Original Image 3.2



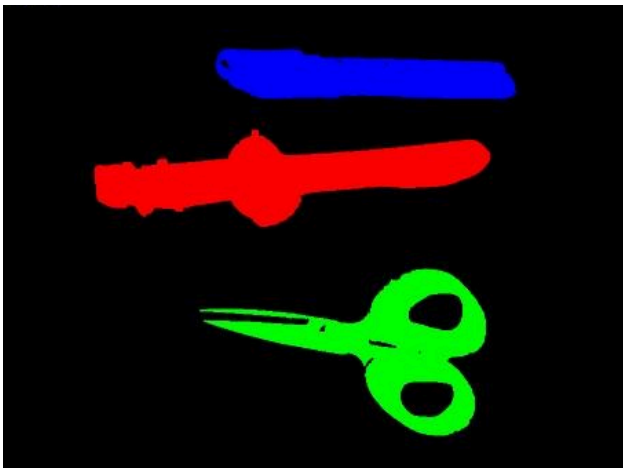
Region Map 3.2

Original Image 3.3



Region Map 3.3

Original Image 3.4



Region Map 3.4

Required images Task 3: Original Images & Region Maps Images

I run connected components analysis (cv::connectedComponentsWithStats) on the cleaned image to label each blob. Regions smaller than a minimum area are ignored, regions touching the image border are discarded (they're usually partial objects), and the top-N largest are kept. Each kept region is drawn in a distinct color from a fixed palette, sorted by area so the dominant object stays consistently colored. The system supports multiple regions, enabling multi-object recognition.

Task 4: Compute features for each major region

Image 4.1 Feature vectors:
Region 0: percentFilled=0.761675 aspectRatio=0.0938686 huMoment1=1.12645

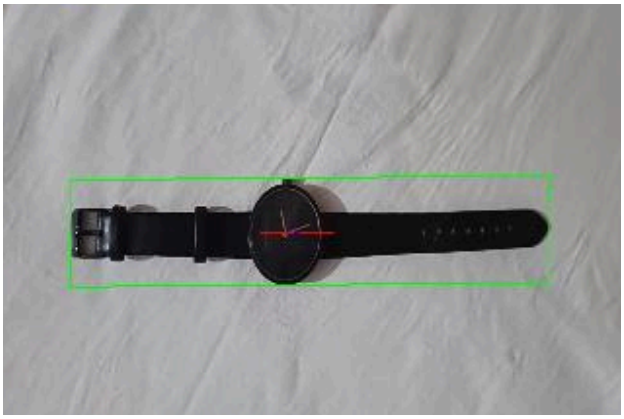


Image 4.2 Feature vectors:
Region 0: percentFilled=0.485539 aspectRatio=0.166837 huMoment1=0.851843

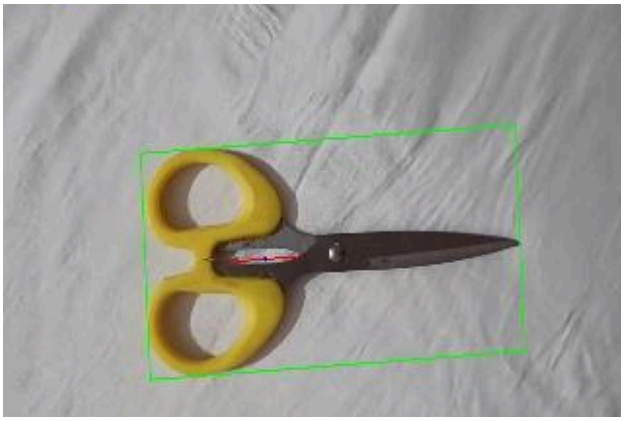


Image 4.3 Feature vectors:
Region 0: percentFilled=0.343022 aspectRatio=0.45619 huMoment1=0.411111

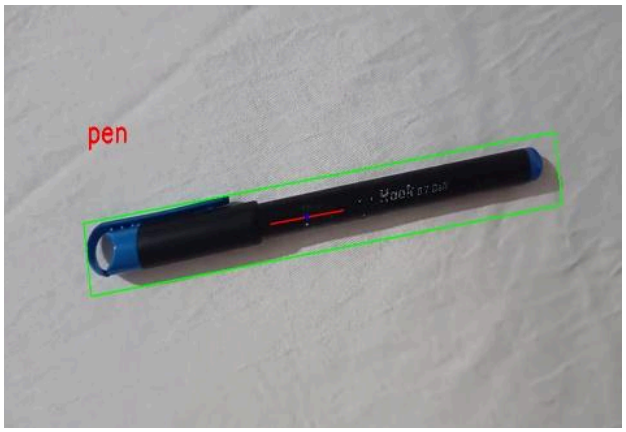
Required images Task 4: Regions showing the axis of least central moment and the oriented bounding box with computed feature vectors

For each region, I compute moments with cv::moments, which gives spatial moments (area, centroid), central moments (translation-invariant), and normalized central moments. From the second central moments, I find the axis of least central moment, $\theta = \frac{1}{2} \cdot \text{atan2}(2 \cdot mu11, mu20 - mu02)$, and rotate region pixels by $-\theta$ to get the oriented bounding box aligned to that axis. I use three translation/scale/rotation-invariant features: percent filled (area/box area), aspect ratio (shorter/longer box side), and the first Hu moment. The oriented box and axis rotate with the object in real time.

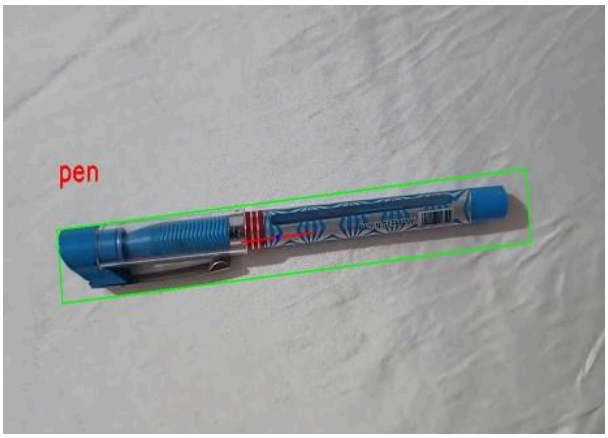
Task 5: Collect training data

Training mode. The system collects labeled training data interactively. When the user presses n, the system computes the feature vector (percent filled, aspect ratio, first Hu moment) for the largest region currently in the frame, then prompts in the terminal for a text label. The label and feature vector are appended as one line to object_db.csv, reusing the CSV format and utility code from Project 2 (label in the first column, features in the remaining columns). To build a robust database, each object is captured several times at different orientations and positions, so the classifier in the next task sees the natural variation in each object's features. The database persists across runs, so training and recognition can happen in separate sessions.

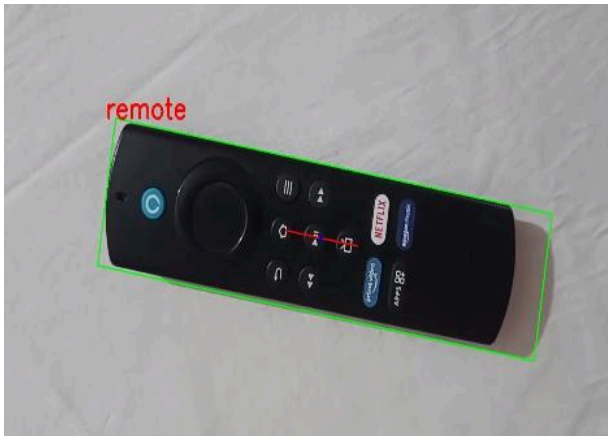
Task 6: Classify new images



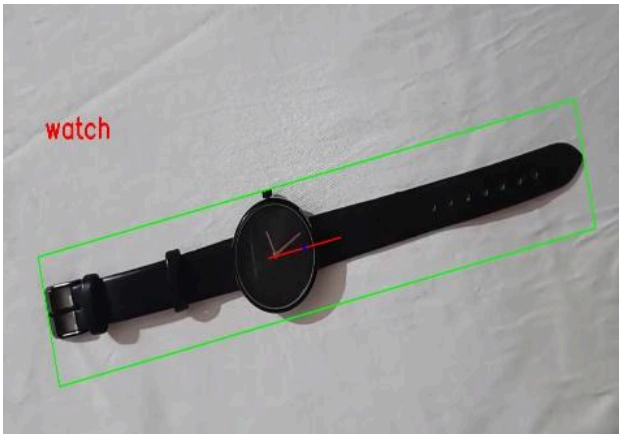
Classified Image 6.1



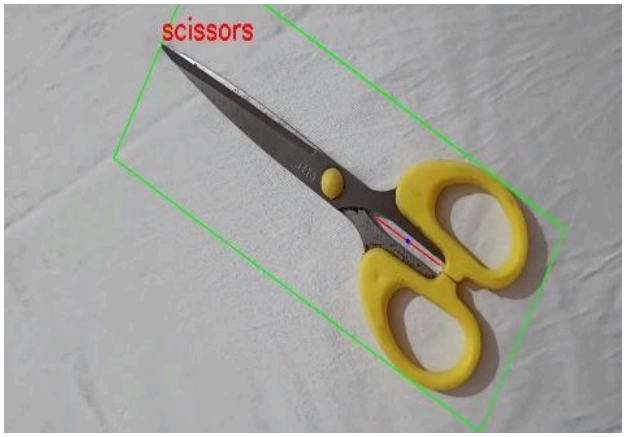
Classified Image 6.2 (Unknown)



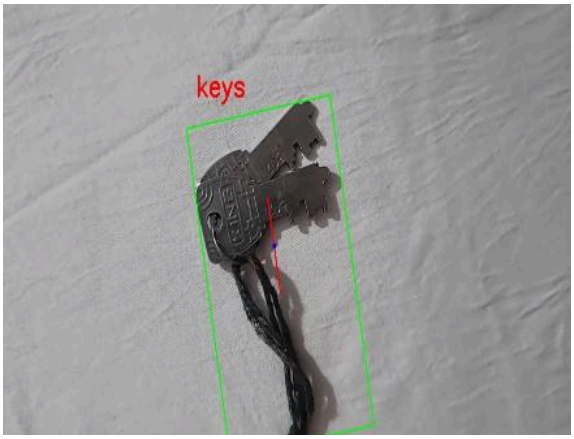
Classified Image 6.3



Classified Image 6.4



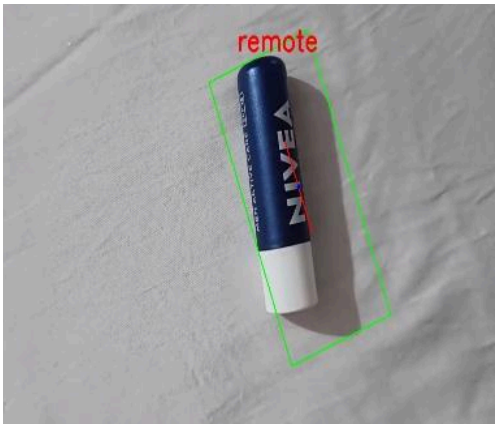
Classified Image 6.5



Classified Image 6.6

Classified Image 6.7

Classified Image 6.8



Classified Image 6.9 (Unknown)

Required images Task 6: Classified Images with assigned labels

Classification. I classified 9 objects, out of which 2 were of the same category (pen), and 7 were known images. New objects are classified using nearest-neighbor matching with scaled Euclidean distance, as specified. At database load time, the system computes the standard deviation of each feature across all training examples. For an unknown object, the distance to each database entry is computed as $d = \sqrt{(\Sigma((a_i - b_i)/\sigma_i)^2)}$, where σ_i is the standard deviation of feature i. Scaling by standard deviation is essential because the features have very different natural ranges: percent filled and aspect ratios are O(1), while the first Hu moment is O(0.001), so without scaling, the Hu moment would be effectively ignored. The unknown object is assigned the label of its nearest neighbor, like here, another pen was classified correctly based on the previous pen's training vectors, but a lip balm was misclassified as a remote. The predicted label is drawn in real time above each detected object in the video stream.

Task 7: Evaluate the performance of your system

```
Confusion Matrix (rows = true, cols = predicted)

true\pred   glasses     keys        pen         remote      scissors    watch
glasses     2           1           0           0           0           0
keys        1           2           0           0           0           0
pen         0           0           3           1           0           0
remote      0           0           0           3           0           0
scissors    0           0           0           0           3           0
watch       0           0           0           0           0           3

Accuracy: 16/19 = 84.2%
```

Image: Screenshot of the confusion matrix and accuracy

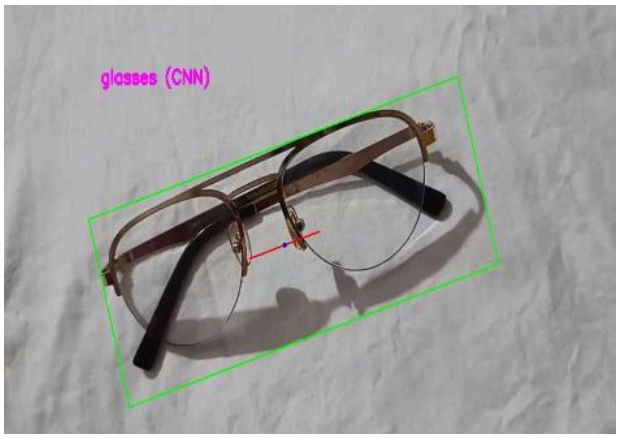
I evaluated the system on 3+ images of each of my 6 objects in different positions and orientations (19 evaluations total). The 6×6 confusion matrix below shows true labels (rows) versus predicted labels (columns). Overall accuracy was **84.2% (16/19)**.

The three errors were not random; they reflect genuine feature overlap. Glasses and keys were confused with each other (one error in each direction): both are small, irregular metallic objects that produce compact, partially-filled blobs with similar percent-filled and aspect-ratio values, so their feature vectors sit close together. One pen was misclassified as a remote, which makes sense because both are elongated rectangular shapes with similar aspect ratios. Objects with distinctive shapes: scissors, watch, and remote were classified perfectly because their features are well-separated from everything else. The errors suggest that adding a feature that better distinguishes compact-but-irregular objects (e.g. more Hu moments capturing finer shape detail) could improve the glasses/keys separation.

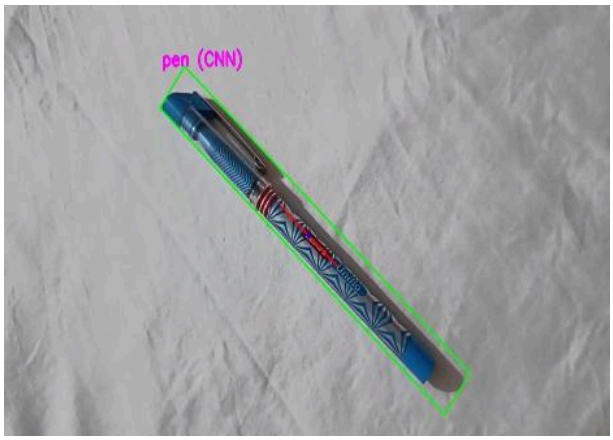
Task 8: Capture a demo of your system working

A video demonstration of the system running and classifying objects in real time is available here: [PRCV Project-3 Demo Video]. It shows the live dashboard with thresholding, region segmentation, feature overlays, and predicted labels updating as objects are moved and rotated.

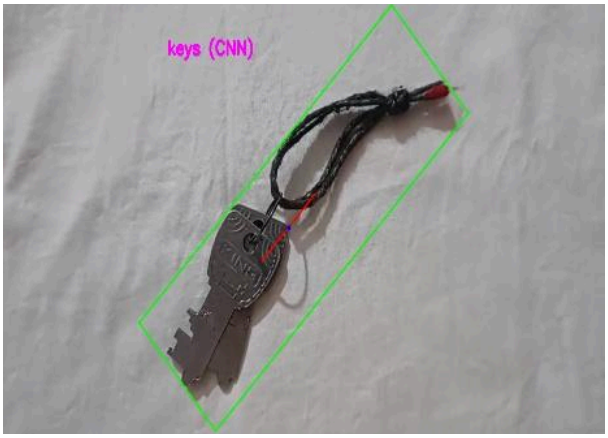
Task 9: Implement a one-shot classification system using image embeddings



CNN-based classified Image 9.1



CNN-based classified Image 9.2



CNN-based classified Image 9.3

The CNN handled the glasses/keys distinction more reliably than the hand-built features. The 512-dimensional ResNet embedding captures finer appearance and shape detail than my three scalar features (percent filled, aspect ratio, first Hu moment), so visually distinct objects that happen to have similar gross shape statistics are still separated well in embedding space. The trade-off is speed: the CNN requires a full network forward pass per classification, whereas the hand-built features are three quick arithmetic computations. For a small set of well-separated objects, the hand-built classifier is faster and adequate (84.2% accuracy), but the CNN is more robust to shape ambiguity and needs only a single training example per object (one-shot).

Extensions

1. **Multiple from-scratch algorithms.**
   Only one of the first four tasks needed to be from scratch; I wrote two: the ISODATA dynamic thresholding (Task 1) and the full morphological filtering suite (erosion, dilation, opening, closing, Task 2).

2. **Saturation-based preprocessing.**
   Beyond a plain intensity threshold, I darken highly saturated pixels in HSV space so colored objects (like bright yellow scissor handles) are correctly captured as foreground rather than being lost to the white background.

3. **Multi-object recognition.**
   The system keeps the top-N regions and labels them simultaneously, each in its own color, rather than handling only a single object.

4. **Custom single-window dashboard GUI.**
   I combined all pipeline views (original, threshold, regions, features) into one composite window.

Images 1 and 2: Screenshots of the GUI (dashboard)

All four pipeline stages are composited into one window in a 2×2 grid: the original feed (top-left), the cleaned threshold (top-right), the colorized region map (bottom-left), and the final feature/recognition view (bottom-right). Three trackbars across the top allow live tuning of the threshold offset, minimum region area, and the number of regions (Top-N) without restarting. In the Features view, each detected object is drawn with its oriented bounding box (green), axis of least central moment (red), centroid, and predicted label.

This replaced an earlier cluttered multi-window layout and improved frame rate by rendering one composite image per frame and running the CNN only on demand.

Reflection

The thing that surprised me most was how much the boring middle steps matter. Thresholding and cleanup aren't the exciting part, but if those are bad, nothing downstream works, your regions get holes, your features come out wrong, and the classifier just guesses. I spent way more time getting the threshold and morphology right than I expected, and once those were solid, everything else kind of fell into place.

Writing the morphological filters from scratch was actually really satisfying once it clicked. Dilation and erosion are such simple ideas, but they cleaned up my scissor blades really nicely.

The features part taught me what "invariant" actually means in practice. I could literally watch the percent-filled number stay the same while I rotated the pen, and watch the oriented box spin to follow it. That made the whole abstract idea concrete.

Comparing my hand-built classifier (84% accuracy) to the ResNet one was eye-opening. My version confused glasses and keys because they're both small blobby metal things with similar shape stats, but the deep network handled them fine because it sees way more detail than three numbers ever could. It made me appreciate both sides, my simple features are fast and good enough for distinct shapes, but the CNN is just way more robust.

The other lesson was performance. My first version had like five separate windows and ran the network every single frame, and it was unusably laggy. Switching to one combined window and only running the CNN when I ask for it made it smooth. Building in Release mode instead of Debug also made a massive difference for the pixel loops.

Acknowledgement

Course materials: I used Prof. Bruce Maxwell's csv_util.cpp/.h for reading and writing the object database, and his utilities.cpp (the getEmbedding and prepEmbeddingImage functions) for

the CNN embedding step in Task 9, with attribution. The ResNet18 ONNX model was provided with the course.

Documentation: I used the OpenCV docs (docs.opencv.org) for cv::moments, cv::HuMoments, connectedComponentsWithStats, the DNN module, and the HighGUI trackbar/mouse functions. I also looked up how the Qt-only createButton isn't available in the prebuilt Windows OpenCV, which is why I built my dashboard with trackbars and a composited image instead.

LLM usage: I wrote, compiled, and ran every line of code myself. All design decisions, like objects, features, weights, and GUI layout, were mine. I used Claude as a learning and debugging assistant. Specifically: explaining things like the axis-of-least-central-moment math, why scaled Euclidean distance is needed when features have different ranges, and how to combine the views into one window. I also used it to debug issues like my classifier not loading the database until I added a reload, and getting the Release build running.